1D ARRAY PROBLEMS — 15

1. Find Maximum and Minimum

Problem: Given an integer array, find its maximum and minimum elements.

Input / Output:
Input: first line N, second line N integers.
Output: print maximum and minimum.

Sample Input:

```
5
10 4 7 2 15
```

Sample Output:

```
Maximum = 15
Minimum = 2
```

Approach: Traverse once. Maintain max and min. Initialize both with the first element and update them for every remaining element.

Java Solution:

```
import java.util.*;
class Main {
public static void main(String[] args) {
Scanner sc = new Scanner(System.in);
int n = sc.nextInt();
int[] a = new int[n];
for (int i = 0; i < n; i++) a[i] = sc.nextInt();

int max = a[0], min = a[0];
for (int i = 1; i < n; i++) {
max = Math.max(max, a[i]);
min = Math.min(min, a[i]);
}
System.out.println("Maximum = " + max);
System.out.println("Minimum = " + min);
}
}
```

Complexity: O(N) time, O(1) extra space

2. Sum and Average of Array

Problem: Find the sum and average of N integers. Print the average to two decimal places.

Input / Output:
Input: N followed by N integers.
Output: sum and average.

Sample Input:

```
5
10 20 30 40 50
```

Sample Output:

```
Sum = 150
Average = 30.00
```

Approach: Accumulate the sum using long to reduce overflow risk. Divide by (double)n for the average.

Java Solution:

```
import java.util.*;
class Main {
public static void main(String[] args) {
Scanner sc = new Scanner(System.in);
int n = sc.nextInt();
long sum = 0;
for (int i = 0; i < n; i++) sum += sc.nextInt();

System.out.println("Sum = " + sum);
System.out.printf("Average = %.2f%n", (double) sum / n);
}
}
```

Complexity: O(N) time, O(1) extra space

3. Reverse an Array

Problem: Reverse the given integer array in place.

Input / Output:
Input: N followed by N integers.
Output: reversed array.

Sample Input:

```
5
1 2 3 4 5
```

Sample Output:

```
5 4 3 2 1
```

Approach: Use two pointers: left at 0 and right at N-1. Swap while left < right.

Java Solution:

```
import java.util.*;
class Main {
public static void main(String[] args) {
Scanner sc = new Scanner(System.in);
int n = sc.nextInt();
int[] a = new int[n];
for (int i = 0; i < n; i++) a[i] = sc.nextInt();

for (int l = 0, r = n - 1; l < r; l++, r--) {
int temp = a[l]; a[l] = a[r]; a[r] = temp;
}

for (int x : a) System.out.print(x + " ");
}
}
```

Complexity: O(N) time, O(1) extra space

4. Second Largest Element

Problem: Find the second largest distinct element in an integer array. If it does not exist, print -1.

Input / Output:
Input: N followed by N integers.
Output: second largest distinct value or -1.

Sample Input:

```
6
10 5 20 20 8 15
```

Sample Output:

```
15
```

Approach: Track the largest and second-largest distinct values in one pass. Update second when a value is between them.

Java Solution:

```
import java.util.*;
class Main {
public static void main(String[] args) {
Scanner sc = new Scanner(System.in);
int n = sc.nextInt();
long first = Long.MIN_VALUE, second = Long.MIN_VALUE;

for (int i = 0; i < n; i++) {
long x = sc.nextLong();
if (x > first) {
second = first;
first = x;
} else if (x > second && x < first) {
second = x;
}
}
System.out.println(second == Long.MIN_VALUE ? -1 : second);
}
}
```

Complexity: O(N) time, O(1) extra space

5. Count Even and Odd Numbers

Problem: Count how many elements are even and how many are odd.

Input / Output:
Input: N followed by N integers.
Output: even count and odd count.

Sample Input:

```
6
1 2 4 7 9 10
```

Sample Output:

```
Even = 3
Odd = 3
```

Approach: For each number, use x % 2 == 0 to classify it as even; otherwise it is odd.

Java Solution:

```
import java.util.*;
class Main {
public static void main(String[] args) {
Scanner sc = new Scanner(System.in);
int n = sc.nextInt(), even = 0, odd = 0;
for (int i = 0; i < n; i++) {
if (sc.nextInt() % 2 == 0) even++;
else odd++;
}
System.out.println("Even = " + even);
System.out.println("Odd = " + odd);
}
}
```

Complexity: O(N) time, O(1) extra space

6. Linear Search

Problem: Find the first index of a target value in an integer array. Print -1 if absent.

Input / Output:
Input: N, N integers, target.
Output: first index or -1.

Sample Input:

```
5
10 20 30 40 50
30
```

Sample Output:

```
2
```

Approach: Scan from left to right. The first matching element is the answer.

Java Solution:

```
import java.util.*;
class Main {
public static void main(String[] args) {
Scanner sc = new Scanner(System.in);
int n = sc.nextInt();
int[] a = new int[n];
for (int i = 0; i < n; i++) a[i] = sc.nextInt();
int target = sc.nextInt();

int index = -1;
for (int i = 0; i < n; i++) {
if (a[i] == target) { index = i; break; }
}
System.out.println(index);
}
}
```

Complexity: O(N) time, O(1) extra space

7. Frequency of Each Element

Problem: Print the frequency of every distinct integer in the order in which it first appears.

Input / Output:
Input: N followed by N integers.
Output: each distinct value and its frequency.

Sample Input:

```
7
2 3 2 5 3 2 5
```

Sample Output:

```
2 -> 3
3 -> 2
5 -> 2
```

Approach: Use a HashMap to count occurrences. Use a LinkedHashMap to preserve first-appearance order.

Java Solution:

```
import java.util.*;
class Main {
public static void main(String[] args) {
Scanner sc = new Scanner(System.in);
int n = sc.nextInt();
Map<Integer,Integer> freq = new LinkedHashMap<>();

for (int i = 0; i < n; i++) {
int x = sc.nextInt();
freq.put(x, freq.getOrDefault(x, 0) + 1);
}

for (var e : freq.entrySet())
System.out.println(e.getKey() + " -> " + e.getValue());
}
}
```

Complexity: O(N) average time, O(K) space

8. Remove Duplicates from Sorted Array

Problem: Given a sorted integer array, remove duplicates in place and print the unique elements.

Input / Output:
Input: N followed by a sorted array.
Output: unique elements.

Sample Input:

```
8
1 1 2 2 2 3 4 4
```

Sample Output:

```
1 2 3 4
```

Approach: Use a write pointer. Whenever a[i] differs from the previous unique element, write it at the next position.

Java Solution:

```
import java.util.*;
class Main {
public static void main(String[] args) {
Scanner sc = new Scanner(System.in);
int n = sc.nextInt();
int[] a = new int[n];
for (int i = 0; i < n; i++) a[i] = sc.nextInt();

if (n == 0) return;
int k = 1;
for (int i = 1; i < n; i++)
if (a[i] != a[k - 1]) a[k++] = a[i];

for (int i = 0; i < k; i++) System.out.print(a[i] + " ");
}
}
```

Complexity: O(N) time, O(1) extra space

9. Move All Zeros to End

Problem: Move all zero values to the end while preserving the relative order of non-zero elements.

Input / Output:
Input: N followed by N integers.
Output: transformed array.

Sample Input:

```
7
0 1 0 3 12 0 5
```

Sample Output:

```
1 3 12 5 0 0 0
```

Approach: Maintain a position for the next non-zero value. First copy all non-zero values forward, then fill the remaining positions with zero.

Java Solution:

```
import java.util.*;
class Main {
public static void main(String[] args) {
Scanner sc = new Scanner(System.in);
int n = sc.nextInt();
int[] a = new int[n];
for (int i = 0; i < n; i++) a[i] = sc.nextInt();

int pos = 0;
for (int x : a)
if (x != 0) a[pos++] = x;
while (pos < n) a[pos++] = 0;

for (int x : a) System.out.print(x + " ");
}
}
```

Complexity: O(N) time, O(1) extra space

10. Left Rotate Array by K

Problem: Rotate an integer array to the left by K positions.

Input / Output:
Input: N, array, K.
Output: rotated array.

Sample Input:

```
5
1 2 3 4 5
2
```

Sample Output:

```
3 4 5 1 2
```

Approach: Use the reversal algorithm: reverse the first K elements, reverse the remaining elements, then reverse the whole array.

Java Solution:

```
import java.util.*;
class Main {
static void reverse(int[] a, int l, int r) {
while (l < r) {
int t = a[l]; a[l] = a[r]; a[r] = t;
l++; r--;
}
}

public static void main(String[] args) {
Scanner sc = new Scanner(System.in);
int n = sc.nextInt();
int[] a = new int[n];
for (int i = 0; i < n; i++) a[i] = sc.nextInt();
int k = sc.nextInt();
k %= n;

reverse(a, 0, k - 1);
reverse(a, k, n - 1);
reverse(a, 0, n - 1);

for (int x : a) System.out.print(x + " ");
}
}
```

Complexity: O(N) time, O(1) extra space

11. Maximum Subarray Sum — Kadane

Problem: Find the maximum sum of a contiguous subarray.

Input / Output:
Input: N followed by N integers.
Output: maximum contiguous subarray sum.

Sample Input:

```
9
-2 1 -3 4 -1 2 1 -5 4
```

Sample Output:

```
6
```

Approach: Kadane's algorithm keeps the best subarray ending at the current position. At each element choose between starting fresh or extending the current sum.

Java Solution:

```
import java.util.*;
class Main {
public static void main(String[] args) {
Scanner sc = new Scanner(System.in);
int n = sc.nextInt();
long current = 0, best = Long.MIN_VALUE;

for (int i = 0; i < n; i++) {
long x = sc.nextLong();
current = Math.max(x, current + x);
best = Math.max(best, current);
}
System.out.println(best);
}
}
```

Complexity: O(N) time, O(1) extra space

12. Two Sum

Problem: Find two indices whose values add up to a target. Print the first pair found.

Input / Output:
Input: N, N integers, target.
Output: two indices or -1 -1.

Sample Input:

```
5
2 7 11 15 3
9
```

Sample Output:

```
0 1
```

Approach: Store previously seen values in a HashMap. For x, search for target - x before storing x.

Java Solution:

```
import java.util.*;
class Main {
public static void main(String[] args) {
Scanner sc = new Scanner(System.in);
int n = sc.nextInt();
int[] a = new int[n];
for (int i = 0; i < n; i++) a[i] = sc.nextInt();
int target = sc.nextInt();

Map<Integer,Integer> map = new HashMap<>();
for (int i = 0; i < n; i++) {
int need = target - a[i];
if (map.containsKey(need)) {
System.out.println(map.get(need) + " " + i);
return;
}
map.put(a[i], i);
}
System.out.println("-1 -1");
}
}
```

Complexity: O(N) average time, O(N) space

13. Merge Two Sorted Arrays

Problem: Merge two sorted integer arrays into one sorted array.

Input / Output:
Input: N, first sorted array, M, second sorted array.
Output: merged sorted array.

Sample Input:

```
4
1 3 5 7
3
2 4 6
```

Sample Output:

```
1 2 3 4 5 6 7
```

Approach: Use two pointers, one for each array. Repeatedly choose the smaller current element and append the remainder.

Java Solution:

```
import java.util.*;
class Main {
public static void main(String[] args) {
Scanner sc = new Scanner(System.in);
int n = sc.nextInt();
int[] a = new int[n];
for (int i = 0; i < n; i++) a[i] = sc.nextInt();

int m = sc.nextInt();
int[] b = new int[m];
for (int i = 0; i < m; i++) b[i] = sc.nextInt();

int[] c = new int[n + m];
int i = 0, j = 0, k = 0;
while (i < n && j < m)
c[k++] = a[i] <= b[j] ? a[i++] : b[j++];
while (i < n) c[k++] = a[i++];
while (j < m) c[k++] = b[j++];

for (int x : c) System.out.print(x + " ");
}
}
```

Complexity: O(N+M) time, O(N+M) space

14. Leaders in an Array

Problem: An element is a leader if it is greater than every element to its right. Print leaders from left to right.

Input / Output:
Input: N followed by N integers.
Output: leader elements.

Sample Input:

```
6
16 17 4 3 5 2
```

Sample Output:

```
17 5 2
```

Approach: Scan from right to left while tracking the maximum seen so far. A value greater than that maximum is a leader. Store leaders and print them in reverse.

Java Solution:

```
import java.util.*;
class Main {
public static void main(String[] args) {
Scanner sc = new Scanner(System.in);
int n = sc.nextInt();
int[] a = new int[n];
for (int i = 0; i < n; i++) a[i] = sc.nextInt();

int[] leaders = new int[n];
int k = 0, max = Integer.MIN_VALUE;
for (int i = n - 1; i >= 0; i--) {
if (a[i] > max) {
leaders[k++] = a[i];
max = a[i];
}
}
for (int i = k - 1; i >= 0; i--) System.out.print(leaders[i] + " ");
}
}
```

Complexity: O(N) time, O(N) output space

15. Product of Array Except Self

Problem: For each position, print the product of all elements except the element at that position. Do not use division.

Input / Output:
Input: N followed by N integers.
Output: N products.

Sample Input:

```
4
1 2 3 4
```

Sample Output:

```
24 12 8 6
```

Approach: Use prefix products in the result array, then multiply by suffix products in a reverse pass.

Java Solution:

```
import java.util.*;
class Main {
public static void main(String[] args) {
Scanner sc = new Scanner(System.in);
int n = sc.nextInt();
int[] a = new int[n];
for (int i = 0; i < n; i++) a[i] = sc.nextInt();

long[] ans = new long[n];
long prefix = 1;
for (int i = 0; i < n; i++) {
ans[i] = prefix;
prefix *= a[i];
}

long suffix = 1;
for (int i = n - 1; i >= 0; i--) {
ans[i] *= suffix;
suffix *= a[i];
}

for (long x : ans) System.out.print(x + " ");
}
}
```

Complexity: O(N) time, O(N) output array

2D ARRAY / MATRIX PROBLEMS — 5

16. Matrix Row and Column Sum

Problem: Given an integer matrix, print the sum of every row and every column.

Input / Output:
Input: R C followed by R*C integers.
Output: row sums followed by column sums.

Sample Input:

```
2 3
1 2 3
4 5 6
```

Sample Output:

```
Row sums: 6 15
Column sums: 5 7 9
```

Approach: Use nested loops. Add each matrix[i][j] to rowSum[i] and columnSum[j].

Java Solution:

```
import java.util.*;
class Main {
public static void main(String[] args) {
Scanner sc = new Scanner(System.in);
int r = sc.nextInt(), c = sc.nextInt();
int[][] a = new int[r][c];
int[] row = new int[r], col = new int[c];

for (int i = 0; i < r; i++)
for (int j = 0; j < c; j++) {
a[i][j] = sc.nextInt();
row[i] += a[i][j];
col[j] += a[i][j];
}

System.out.print("Row sums: ");
for (int x : row) System.out.print(x + " ");
System.out.print("\nColumn sums: ");
for (int x : col) System.out.print(x + " ");
}
}
```

Complexity: O(R*C) time, O(R+C) extra space

17. Matrix Transpose

Problem: Given an R x C matrix, print its transpose.

Input / Output:
Input: R C followed by matrix values.
Output: C x R transpose.

Sample Input:

```
2 3
1 2 3
4 5 6
```

Sample Output:

```
1 4
2 5
3 6
```

Approach: The transpose swaps row and column positions: result[j][i] = matrix[i][j].

Java Solution:

```
import java.util.*;
class Main {
public static void main(String[] args) {
Scanner sc = new Scanner(System.in);
int r = sc.nextInt(), c = sc.nextInt();
int[][] a = new int[r][c];

for (int i = 0; i < r; i++)
for (int j = 0; j < c; j++) a[i][j] = sc.nextInt();

for (int j = 0; j < c; j++) {
for (int i = 0; i < r; i++)
System.out.print(a[i][j] + " ");
System.out.println();
}
}
}
```

Complexity: O(R*C) time, O(1) extra space beyond input

18. Matrix Multiplication

Problem: Multiply two compatible integer matrices A and B.

Input / Output:
Input: dimensions and values of A, then dimensions and values of B.
Output: A*B.

Sample Input:

```
2 2
1 2
3 4
2 2
5 6
7 8
```

Sample Output:

```
19 22
43 50
```

Approach: If A is R x K and B is K x C, result is R x C. For each result cell, compute the dot product of one row of A and one column of B.

Java Solution:

```
import java.util.*;
class Main {
public static void main(String[] args) {
Scanner sc = new Scanner(System.in);
int r1 = sc.nextInt(), k = sc.nextInt();
int[][] a = new int[r1][k];
for (int i = 0; i < r1; i++)
for (int j = 0; j < k; j++) a[i][j] = sc.nextInt();

int r2 = sc.nextInt(), c = sc.nextInt();
int[][] b = new int[r2][c];
for (int i = 0; i < r2; i++)
for (int j = 0; j < c; j++) b[i][j] = sc.nextInt();

if (k != r2) { System.out.println("Invalid dimensions"); return; }

long[][] ans = new long[r1][c];
for (int i = 0; i < r1; i++)
for (int j = 0; j < c; j++)
for (int x = 0; x < k; x++)
ans[i][j] += (long)a[i][x] * b[x][j];

for (long[] row : ans) {
for (long x : row) System.out.print(x + " ");
System.out.println();
}
}
}
```

Complexity: O(R1*K*C) time, O(R1*C) result space

19. Spiral Traversal of Matrix

Problem: Print the elements of a matrix in spiral order.

Input / Output:
Input: R C followed by matrix.
Output: spiral sequence.

Sample Input:

```
3 4
1 2 3 4
5 6 7 8
9 10 11 12
```

Sample Output:

```
1 2 3 4 8 12 11 10 9 5 6 7
```

Approach: Maintain four boundaries: top, bottom, left, right. Traverse the top row, right column, bottom row, and left column, shrinking boundaries after each pass.

Java Solution:

```
import java.util.*;
class Main {
public static void main(String[] args) {
Scanner sc = new Scanner(System.in);
int r = sc.nextInt(), c = sc.nextInt();
int[][] a = new int[r][c];
for (int i = 0; i < r; i++)
for (int j = 0; j < c; j++) a[i][j] = sc.nextInt();

int top=0, bottom=r-1, left=0, right=c-1;
while (top <= bottom && left <= right) {
for (int j=left; j<=right; j++) System.out.print(a[top][j] + " ");
top++;
for (int i=top; i<=bottom; i++) System.out.print(a[i][right] + " ");
right--;
if (top <= bottom) {
for (int j=right; j>=left; j--) System.out.print(a[bottom][j] + " ");
bottom--;
}
if (left <= right) {
for (int i=bottom; i>=top; i--) System.out.print(a[i][left] + " ");
left++;
}
}
}
}
```

Complexity: O(R*C) time, O(1) extra space

20. Jagged Array — Row with Maximum Sum

Problem: Given a Java jagged integer array with different row lengths, find the row having the maximum sum. Print its 0-based row index and sum.

Input / Output:
Input: R, then for each row its length followed by its elements.
Output: row index and maximum sum.

Sample Input:

```
3
3 1 2 3
2 10 5
4 1 1 1 1
```

Sample Output:

```
Row = 1
Maximum Sum = 15
```

Approach: A jagged array is an array of arrays. Each row may have a different length. Iterate through each row independently and calculate its sum.

Java Solution:

```
import java.util.*;
class Main {
public static void main(String[] args) {
Scanner sc = new Scanner(System.in);
int r = sc.nextInt();
int[][] a = new int[r][];

for (int i = 0; i < r; i++) {
int len = sc.nextInt();
a[i] = new int[len];
for (int j = 0; j < len; j++) a[i][j] = sc.nextInt();
}

int bestRow = -1, bestSum = Integer.MIN_VALUE;
for (int i = 0; i < r; i++) {
int sum = 0;
for (int x : a[i]) sum += x;
if (sum > bestSum) {
bestSum = sum;
bestRow = i;
}
}
System.out.println("Row = " + bestRow);
System.out.println("Maximum Sum = " + bestSum);
}
}
```

Complexity: O(total elements) time, O(total elements) input space

Pattern Coverage

Basic traversal: Maximum/minimum, sum/average, even/odd

Two pointers: Reverse, remove duplicates

Hashing: Frequency, Two Sum

Array transformation: Move zeros, rotation

Prefix/suffix: Product except self

Dynamic programming: Kadane maximum subarray

Sorted arrays: Merge two sorted arrays

Right-to-left scan: Array leaders

2D traversal: Rows/columns, transpose, spiral

Matrix mathematics: Matrix multiplication

Jagged arrays: Different row lengths and row-wise processing

Teaching note: These problems intentionally cover recurring patterns rather than merely listing array operations. For beginner training, problems 1–6 build traversal fundamentals; 7–15 introduce hashing, two pointers, rotation, Kadane, merging, and prefix/suffix techniques; 16–20 build core 2D and jagged-array skills.